Solution: Problem Challenge 1: Count of Subset Sum

Problem Statement

Given a set of positive numbers, find the total number of subsets whose sum is equal to a given number ‘S’.

Example 1:

```
Input: {1, 1, 2, 3}, S=4
Output: 3
The given set has '3' subsets whose sum is '4': {1, 1, 2}, {1, 3}, {1, 3}
Note that we have two similar sets {1, 3}, because we have two '1' in our input.
```

Example 2:

```
Input: {1, 2, 7, 1, 5}, S=9
Output: 3
The given set has '3' subsets whose sum is '9': {2, 7}, {1, 7, 1}, {1, 2, 1, 5}
```

Basic Solution

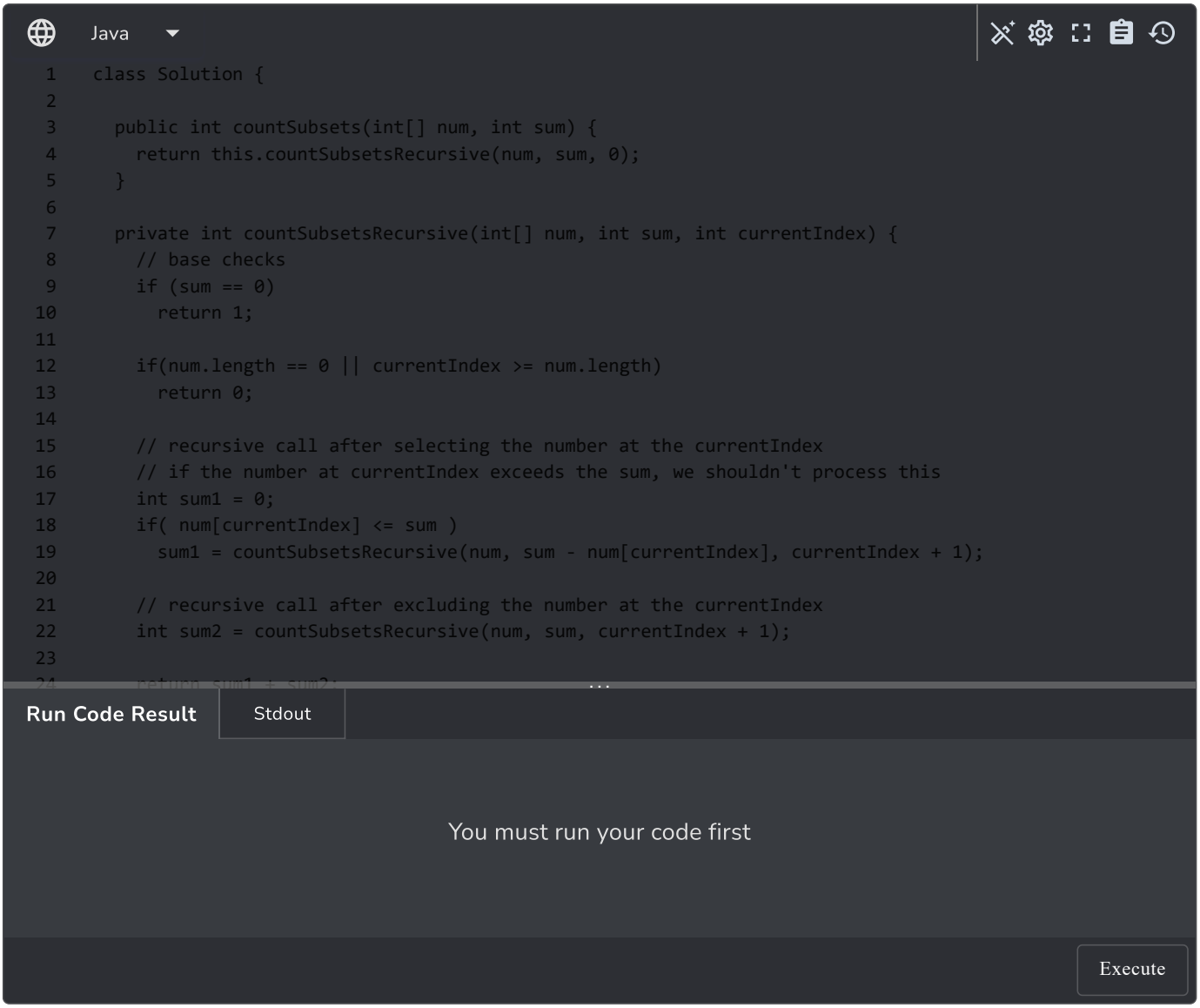
This problem follows the **0/1 Knapsack pattern** and is quite similar to `Subset Sum`. The only difference in this problem is that we need to count the number of subsets, whereas in `Subset Sum` we only wanted to know if a subset with the given sum existed.

A basic brute-force solution could be to try all subsets of the given numbers to count the subsets that have a sum equal to ‘S’. So our brute-force algorithm will look like:

```
for each number 'i'
  create a new set which includes number 'i' if it does not exceed 'S', and recursively
      process the remaining numbers and sum
  create a new set without number 'i', and recursively process the remaining numbers
return the count of subsets who has a sum equal to 'S'
```

Code

Here is the code for the brute-force solution:



Time and Space Complexity

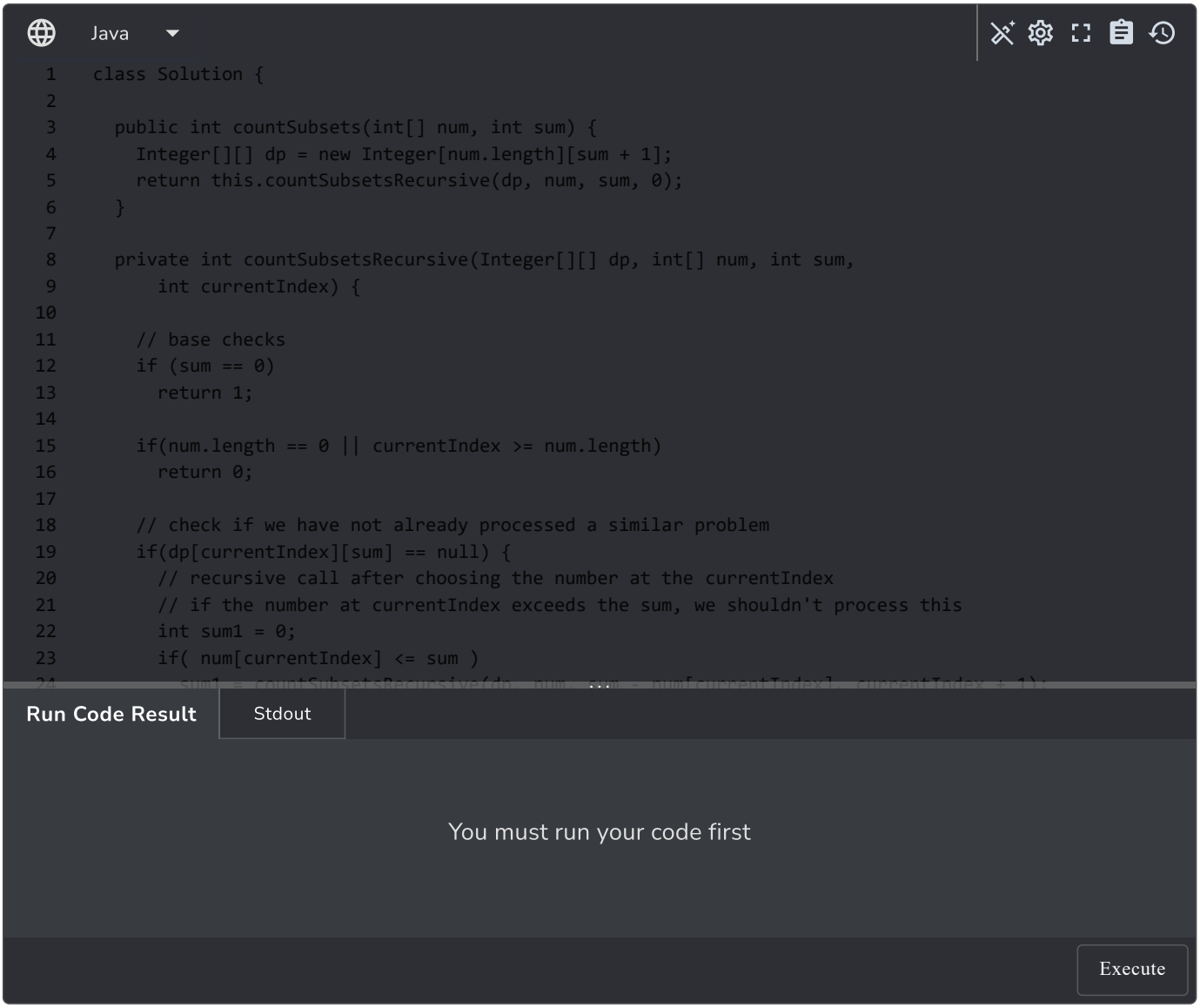
The time complexity of the above algorithm is exponential O(2^n), where ‘n’ represents the total number. The space complexity is O(n), this memory is used to store the recursion stack.

Top-down Dynamic Programming with Memoization

We can use memoization to overcome the overlapping sub-problems. We will be using a two-dimensional array to store the results of solved sub-problems. As mentioned above, we need to store results for every subset and for every possible sum.

Code

Here is the code:



Bottom-up Dynamic Programming

We will try to find if we can make all possible sums with every subset to populate the array `db[TotalNumbers][S+1]`.

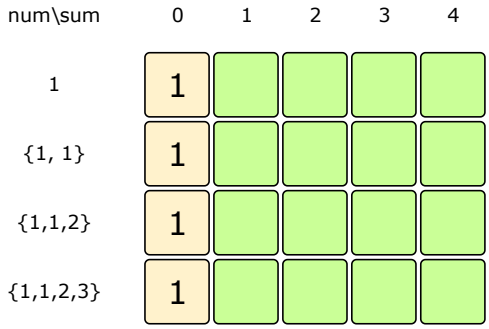
So, at every step we have two options:

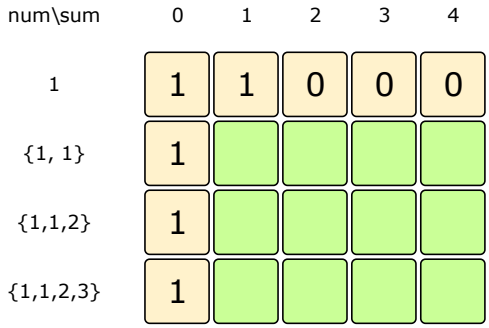
1. Exclude the number. Count all the subsets without the given number up to the given sum => `dp[index-1][sum]`
2. Include the number if its value is not more than the 'sum'. In this case, we will count all the subsets to get the remaining sum => `dp[index-1][sum-num[index]]`

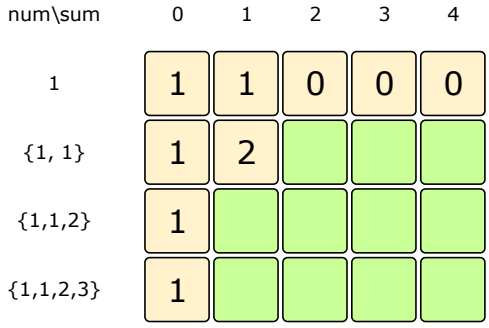
To find the total sets, we will add both of the above two values:

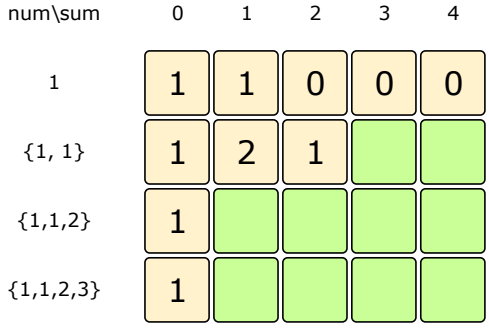
```
dp[index][sum] = dp[index-1][sum] + dp[index-1][sum-num[index]])
```

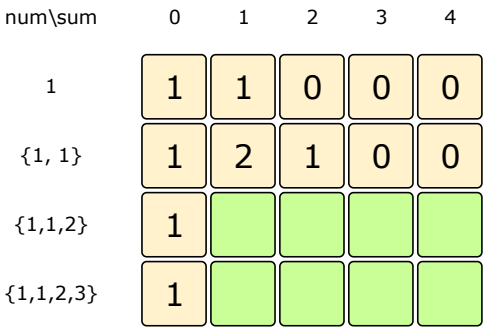
Let's start with our base case of size zero:

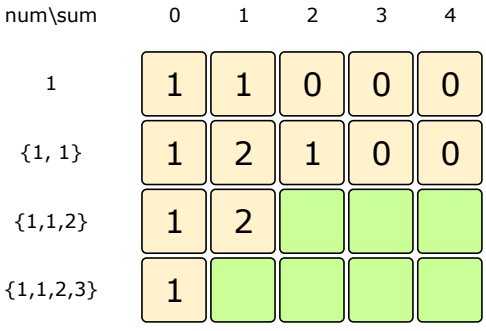


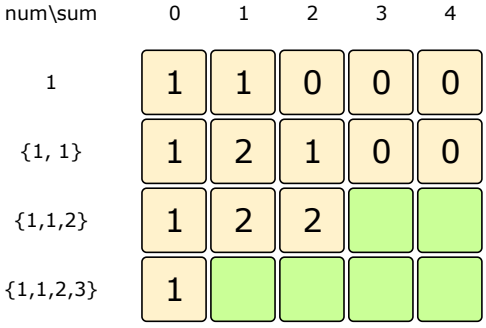


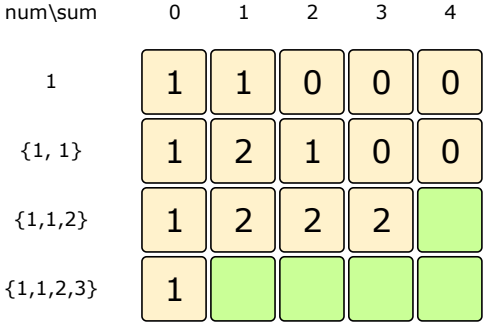


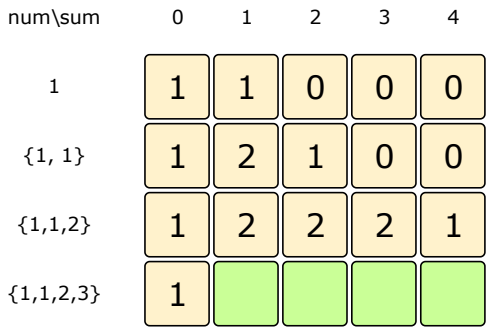


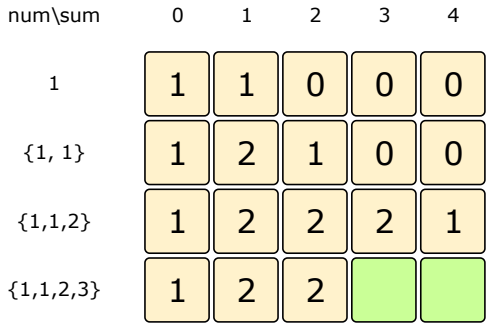

num\sum
0
1
2
3
4
1
1
1
0
0
0
{1, 1}
1
2
1
0
0
{1,1,2}
1
{1,1,2,3}
1

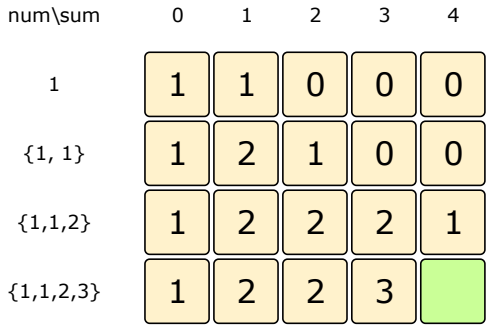

num\sum
0
1
2
3
4
1
1
1
0
0
0
{1, 1}
1
2
1
0
0
{1,1,2}
1
2
{1,1,2,3}
1

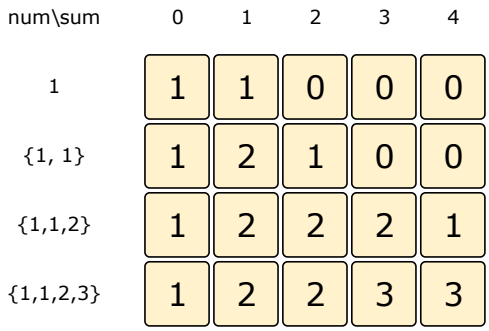

num\sum
0
1
2
3
4
1
1
1
0
0
0
{1, 1}
1
2
1
0
0
{1,1,2}
1
2
2
{1,1,2,3}
1


num\sum
0
1
2
3
4
1
1
1
0
0
0
{1, 1}
1
2
1
0
0
{1,1,2}
1
2
2
2
{1,1,2,3}
1









Code

Here is the code for our bottom-up dynamic programming approach:

```java
class Solution {

  public int countSubsets(int[] num, int sum) {
```

```
    int n = num.length;
    int[][] dp = new int[n][sum + 1];

    // populate the sum=0 columns, as we will always have an empty set for zero sum
    for(int i=0; i < n; i++)
      dp[i][0] = 1;

    // with only one number, we can form a subset only when the required sum is equal
    // to its value
    for(int s=1; s <= sum ; s++) {
      dp[0][s] = (num[0] == s ? 1 : 0);
    }

    // process all subsets for all sums
    for(int i=1; i < num.length; i++) {
      for(int s=1; s <= sum; s++) {
        // exclude the number
        dp[i][s] = dp[i-1][s];
        // include the number, if it does not exceed the sum
        if(s >= num[i])
```

Run Code Result | Stdout

You must run your code first

Execute

Time and Space Complexity

The above solution has the time and space complexity of O(N*S), where 'N' represents total numbers and 'S' is the desired sum.

Challenge

Can we improve our bottom-up DP solution even further? Can you find an algorithm that has O(S) space complexity?

Similar to the space optimized solution for 0/1 Knapsack!

Java

```
class Solution {
  public int countSubsets(int[] num, int sum) {
    int n = num.length;
    int[] dp = new int[sum + 1];
    dp[0] = 1;

    // with only one number, we can form a subset only when the required sum is equal
    // to its value
    for(int s=1; s <= sum ; s++) {
      dp[s] = (num[0] == s ? 1 : 0);
    }
```

```
12
13        // process all subsets for all sums
14        for(int i=1; i < num.length; i++) {
15          for(int s=sum; s >= 0; s--) {
16            if(s >= num[i])
17              dp[s] += dp[s-num[i]];
18          }
19        }
20
21        return dp[sum];
22      }
23      public static void main(String[] args) {
24        Solution ss = new Solution();
```

Run Code Result | Stdout

You must run your code first

Execute

← Previous

Problem Challenge 1: Count Of Subset Sum (Hard)

Next →

Problem Challenge 2: Target Sum (Hard)

Mark as Completed